

Distributional semantics

(and word and sentence embeddings)

(CC-BY: Maria Skeppstedt)

Distributional semantics and word embedding models

Word embeddings and distributional semantics are built on the “distributional hypothesis”, i.e., that words that occur in similar language contexts also have a similar meaning.

Distributional semantics

Which of the words in bold are more or less similar?

- ▶ I drank a cup of **tea** in the morning.
- ▶ I had a cup of **coffee** in the break.
- ▶ At breakfast, I usually drink **coffee**.
- ▶ I drank **juice** to my morning porridge.
- ▶ A usually **sing** on Fridays.

Distributional semantics

- ▶ I drank a cup of **tea** in the morning.
- ▶ I had a cup of **coffee** in the break.
- ▶ At breakfast, I usually drink **coffee**.
- ▶ I drank **juice** to my morning porridge.
- ▶ A usually **sing** on Fridays.

Words occurring in similar contexts are paradigmatically similar.

This can be encoded through word embeddings.

Word embedding models

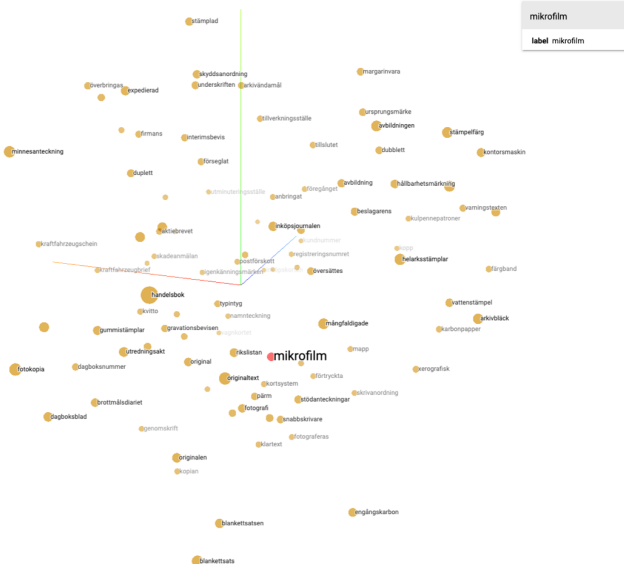
- ▶ Word embeddings models consist of a large number of multi-dimensional vectors, where each vector represents a word.
- ▶ The vectors can be seen as points in a multi-dimensional space.
- ▶ How close two vectors are in the vectors space, thereby encodes the similarity of the words, i.e. their paradigmatic relations.
- ▶ There are many different ways of constructing word embedding vectors.

Word embedding vectors

Often, the word embedding vectors have 100 to 300 elements.

```
tea = [-0.0055 0.0475 0.1709 ... 0.0557 0.0317 -0.0541]  
coffee = [0.0299 0.0274 0.1440 ... 0.0673 0.0079 0.0070]  
juice = [-0.0566 0.1543 0.1848 ... 0.0376 0.0374 -0.0148]  
sing = [-0.0671 -0.1263 0.1155 ... -0.0285 -0.0754 -0.0350]
```

3D visualisation example



2D visualisation example

The multi-dimensional vectors can be project down to just two dimensions, to provide (an approximate) visualisation of the similarity between the words.



Cosine similarity

Cosine similarity can be computed using the dot products and the norms of the vectors.

$$\cos(\mathbf{v}, \mathbf{u}) = \frac{\mathbf{v} \cdot \mathbf{u}}{||\mathbf{v}|| \cdot ||\mathbf{u}||} \quad (1)$$

The norm can be obtained by:

```
1 np.linalg.norm
```

Similar



- ▶ ('coffee', 'coffee') 1.0000001
- ▶ ('sing', 'sings') 0.9463291
- ▶ ('sing', 'singing') 0.865106
- ▶ ('coffee', 'tea') 0.6656748
- ▶ ('good', 'bad') 0.6035023

Not so similar



- ▶ ('juice', 'milk') 0.4736316
- ▶ ('sing', 'shout') 0.41401654
- ▶ ('shout', 'blue') 0.030636346
- ▶ ('juice', 'sing') -0.061370883

Cosine similarity

Note that a cosine similarity of **1** means identical.

A small cosine similarity means that two words are **not** similar.

- ▶ ('coffee', 'coffee') 1.0000001
- ▶ ('sing', 'sings') 0.9463291
- ▶ ('sing', 'singing') 0.865106
- ▶ ('coffee', 'tea') 0.6656748
- ▶ ('good', 'bad') 0.6035023
- ▶ ('juice', 'milk') 0.4736316
- ▶ ('sing', 'shout') 0.41401654
- ▶ ('shout', 'blue') 0.030636346
- ▶ ('juice', 'sing') -0.061370883

Cosine distance

Note that in Scikit-learn, **cosine distance** is sometimes used as a measure. That is defined as $1 - \text{cosinesimilarity}$

Sentence embeddings

It is also possible to create embeddings on larger parts of texts than words.

Also for these embeddings, there are several different possible methods.

Sentence embeddings

Same format, but encodes a sentence.

```
['my', 'dog', 'likes', 'eating']  
[-0.061375 -0.0413      0.143575 ...  
-0.009625  0.01235  -0.0293  ]
```

```
['my', 'cat', 'likes', 'playing']  
[-0.0686  -0.0732   0.10485 ...  
-0.002      0.00115  -0.008225]
```

```
['I', 'drink', 'coffee', 'in', 'the', 'morning']  
[-0.03206 -0.05084  0.0579   ...  
 0.00822 -0.01424  0.00014]
```

```
['I', 'drink', 'tea', 'in', 'the', 'evening']  
[-0.03892   -0.05435999  0.06396    ...  
-0.00298 -0.0114  -0.0211  ]
```

Sentence embeddings

Also use cosine similarity for determining how similar they are:

```
(['my', 'dog', 'likes', 'eating'],  
 ['my', 'cat', 'likes', 'playing'])
```

Cosine similarity: 0.77942646

```
(['I', 'drink', 'tea', 'in', 'the', 'evening'],  
 ['I', 'drink', 'coffee', 'in', 'the', 'morning'])
```

Cosine similarity: 0.90893215

```
(['my', 'dog', 'likes', 'eating'],  
 ['I', 'drink', 'coffee', 'in', 'the', 'morning'])
```

Cosine similarity: 0.27764472

```
(['my', 'dog', 'likes', 'eating'],  
 ['my', 'dog', 'likes', 'eating'])
```

Cosine similarity: 0.9999999